

1. Introducción

El objetivo de esta actividad es crear un pequeño mini juego que consta de tres botones y tres luces LED. La idea del juego es que se enciendan las luces de manera aleatoria y cuando el usuario presione el botón correspondiente esta se apague, se registre un acierto y se encienda otra luz. El juego se detendrá al alcanzar los diez aciertos y sonará una pequeña canción de triunfo o derrota según sea el caso.

Esta actividad es una continuación de la anterior, en donde solamente se utilizaba un LED y un botón. Se trabajó en una nueva etapa del proyecto de entretenimiento, la cual consiste en usar tres LEDs y tres botones de diferentes colores. La idea es que uno de los LEDs se prenda de forma aleatoria y que el usuario tenga que presionar el botón que corresponde al LED que está encendido. Cuando se presiona el botón correcto, el LED se apaga y se enciende otro LED, que es seleccionado de forma aleatoria. Si se presiona un botón que no corresponde, el sistema no hace nada y el LED que estaba prendido se mantiene así hasta que se presione el botón correcto.

2. Objetivo

El objetivo de esta actividad es ampliar el sistema que se había realizado anteriormente para poder utilizar tres LEDs y tres botones.

Cada botón tiene que estar relacionado con un LED específico y el sistema debe poder seleccionar aleatoriamente cuál LED se va a prender. También se busca que el sistema pueda detectar cuando el usuario presiona el botón correcto o uno incorrecto.

3. Materiales

Para realizar la versión actual del circuito se necesitan los siguientes materiales:

Componente	Cantidad	Función
Arduino UNO R3	1	Controlar el funcionamiento del sistema.
Protoboard	1	Conectar los diferentes componentes de una manera más sencilla sin necesidad de soldarlos.
Pulsador (botón)	3	Entrada para que el usuario pueda ingresar su respuesta y cambiar el estado del sistema.
Resistor 10k Ω	3	Limitar la corriente que circula por el botón
Resistor 220 Ω	3	Limitar la corriente que circula por el LED
LED	3	Mostrar visualmente el estado del sistema y mostrar qué opción debe seleccionar el usuario.
Cables jumper	12	Conectar el Arduino al protoboard y conectar a los componentes entre sí.
Cable USB-A a USB-B	1	Conectar el Arduino a la computadora para cargar el programa y alimentar el circuito.

Nota:

En nuestra implementación utilizamos LEDs de diferentes colores para diferenciar de forma clara cada uno de ellos y facilitar el juego para el usuario. También decidimos utilizar solo resistores de 220 Ω en cada LED por simplicidad. Sin embargo, la literatura indica que dependiendo del color del LED, estos operan en un voltaje diferente; por ende si queremos que todos brillen con aproximadamente la misma intensidad, habría que poner valores de resistencia específicos para cada uno (Boylestad y Nashelsky, 2013).

Recomendamos los siguientes valores para operar debajo de 15mA, aunque si algún LED es muy brillante, subir la resistencia puede reducir su intensidad sin dañar el Arduino:

Color de LED	Voltaje típico	Resistencia recomendada
Rojo	2.0 V	220 Ω
Verde	2.2 V	180 Ω o 220 Ω
Azul	3.3 V	120 Ω

4. Metodología

Al iniciar el Arduino, el programa selecciona uno de los tres LEDs de manera aleatoria y lo enciende. El usuario debe observar cuál LED está encendido y presionar el botón que corresponde a ese color. Si el botón es correcto, el programa apaga el LED y selecciona otro LED aleatoriamente. Si el usuario presiona un botón que no corresponde al LED encendido, el sistema no cambia de estado. El LED permanece encendido hasta que se presione el botón correcto.

Primero se colocó el Arduino UNO R3 junto con la protoboard para poder realizar las conexiones. Se utilizó el pin de 5V del Arduino para alimentar la línea positiva de la protoboard y uno de los pines GND para conectar la línea negativa.

4.1 Conexión de los LEDs

Se colocaron los tres LEDs en el protoboard. Los LEDs utilizados fueron uno rojo, uno azul y uno verde.

Se debe conectar el ánodo del LED (normalmente la terminal más larga) al puerto del Arduino que controlará dicho LED, mientras que el cátodo (la terminal más corta), se conecta a GND para cerrar el circuito. Cada LED se conecta en serie con un resistor de 220Ω para limitar la corriente y evitar que este pueda dañarse; aunque como se mencionó anteriormente, este valor de resistencia puede variar dependiendo del color del LED y la intensidad del brillo deseado. Dicho resistor puede conectarse entre el ánodo y el Arduino o el cátodo y GND; funciona correctamente en cualquiera de esas opciones.

La conexión realizada fue:

Pin Arduino — Resistor de 220Ω — LED — GND

Los LEDs se conectaron a los pines digitales 11, 12 y 13 del Arduino.

4.2 Conexión de los botones (pulsadores)

Los tres botones se colocaron en la parte central del protoboard. Cada botón tiene una conexión hacia un pin digital diferente del Arduino y otra hacia la alimentación correspondiente. En este circuito se utilizó un resistor de $10k\Omega$. De acuerdo con SparkFun Electronics (s.f.), una resistencia de $10k\Omega$ es el valor ideal para mantener un bajo consumo de corriente sin perder la inmunidad al ruido.

Este resistor conecta la entrada el pin correspondiente de nuestro Arduino con GND, haciendo que el Arduino detecte un estado de LOW(Apagado) cuando el botón no está presionado. El otro lado del botón se conectó a la alimentación de 5V, por esta razón cuando el botón es presionado los 5V llegan al pin del Arduino y este detecta esto como un estado de HIGH(Encendido).

La conexión realizada fue:

5V — Botón — Pin Arduino

Después:

Pin Arduino — Resistor de 10k Ω — GND

Los botones se conectaron a los pines digitales 2, 3 y 4 del Arduino.

La distribución de los componentes quedó de la siguiente manera:

Componente	Pin de Arduino	Función
LED rojo	11	Indicar la opción roja
LED azul	12	Indicar la opción azul
LED verde	13	Indicar la opción verde
Botón rojo	2	Seleccionar el LED rojo
Botón azul	3	Seleccionar el LED azul
Botón verde	4	Seleccionar el LED verde

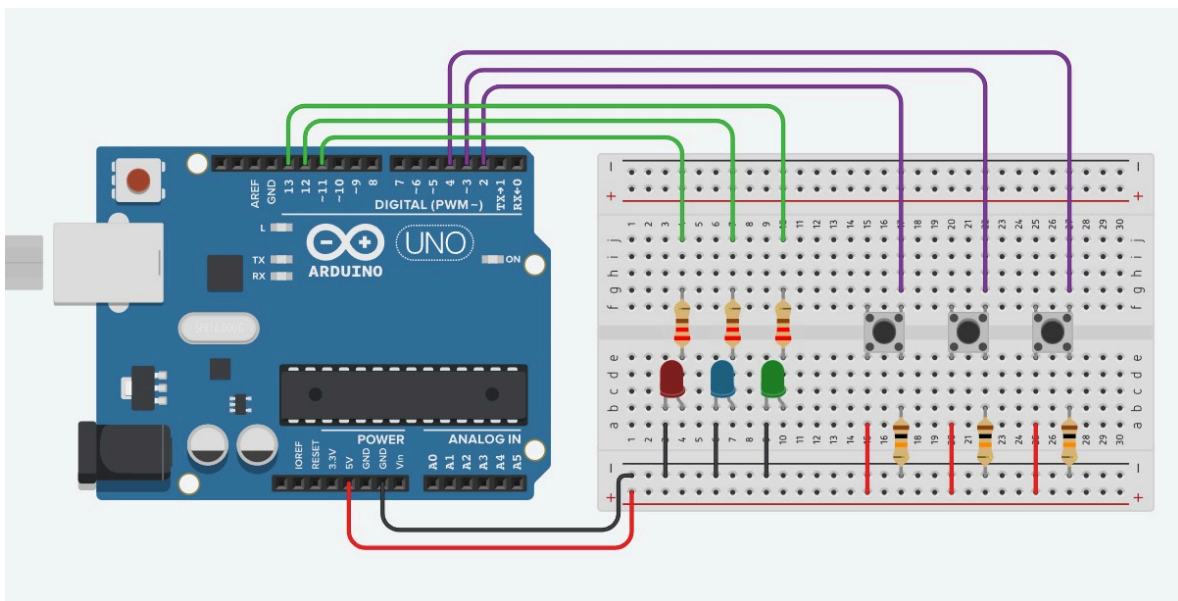
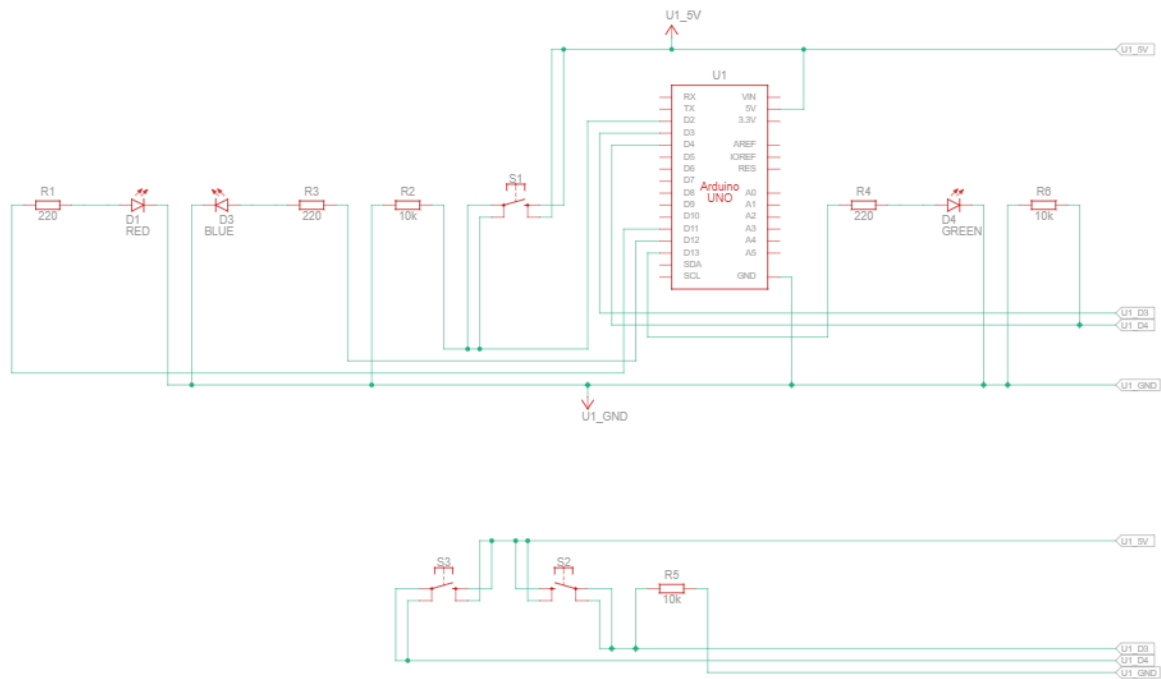
De esta manera cada LED tiene un botón correspondiente y el Arduino puede saber cuál componente fue activado.

4.3 Diagrama de conexión

En la imagen se puede observar el Arduino Uno conectado a la protoboard. También se pueden identificar los tres LEDs, los tres botones, las resistencias y los cables utilizados para realizar las conexiones.

Los cables se acomodaron de forma que fuera más fácil identificar cada conexión y poder revisar el circuito en caso de que algún componente no funcionara correctamente.

La actividad recomienda utilizar Tinkercad para mostrar de forma clara las conexiones del sistema.



5. Código utilizado

```
1 int LED1 = 11;
2 int LED2 = 12;
3 int LED3 = 13;
4 int boton1 = 2;
5 int boton2 = 3;
6 int boton3 = 4;
7 int opcion = 0;
8 int aciertos = 0;
9 int errores = 0;
10 bool presionado;
11
12 //-----
13
14 void setup() {
15     //Iniciamos los LEDs como salidas de nuestro Arduino.
16     pinMode(LED1, OUTPUT);
17     pinMode(LED2, OUTPUT);
18     pinMode(LED3, OUTPUT);
19
20     //Iniciamos los botones como entradas de nuestro Arduino.
21     pinMode(boton1, INPUT);
22     pinMode(boton2, INPUT);
23     pinMode(boton3, INPUT);
24
25     randomSeed(analogRead(A0));
26 }
27
28 //-----
29
30 void loop() {
31
32     opcion = random(1, 4); //Genera el LED que se encenderá de forma random
33     presionado = false; //Inicializamos la variable de control de presionado
34
35     //Encendemos el LED correspondiente al número random generado:
36     if (opcion == 1) digitalWrite(LED1, HIGH);
37     else if (opcion == 2) digitalWrite(LED2, HIGH);
38     else if (opcion == 3) digitalWrite(LED3, HIGH);
39
40     while (!presionado) { //Iniciamos un ciclo aquí, este atrapa el programa aquí mientras el usuario no presione el botón.
41
42         if (digitalRead(boton1) == HIGH) { //Evaluamos si el botón presionado fue el Boton 1.
43             if (opcion == 1) { //Revisamos si el LED que estaba encendido era el 1 o no. En cuyo caso, el usuario acertó.
44                 aciertos++; //Agregamos 1 al contador de aciertos. Este contador se usará en futuras versiones del proyecto.
45                 digitalWrite(LED1, LOW); //Ya que el usuario acertó, apagamos el LED que estaba encendido.
46                 presionado = true; //Cambiamos el estado de la variable que controla nuestro ciclo while para poder salir.
47             }
48             else errores++; //Si el LED que estaba encendido no era el 1, el usuario se equivocó. Agregamos al contador de errores.
49         }
50         else if (digitalRead(boton2) == HIGH) { //Evaluamos si el botón presionado fue el Boton 2.
51             if (opcion == 2) { //Revisamos si el LED que estaba encendido era el 2 o no. En cuyo caso, el usuario acertó.
52                 aciertos++; //Agregamos 1 al contador de aciertos. Este contador se usará en futuras versiones del proyecto.
53                 digitalWrite(LED2, LOW); //Ya que el usuario acertó, apagamos el LED que estaba encendido.
54                 presionado = true; //Cambiamos el estado de la variable que controla nuestro ciclo while para poder salir.
55             }
56             else errores++; //Si el LED que estaba encendido no era el 2, el usuario se equivocó. Agregamos al contador de errores.
57         }
58         else if (digitalRead(boton3) == HIGH) { //Evaluamos si el botón presionado fue el Boton 3.
59             if (opcion == 3) { //Revisamos si el LED que estaba encendido era el 3 o no. En cuyo caso, el usuario acertó.
60                 aciertos++; //Agregamos 1 al contador de aciertos. Este contador se usará en futuras versiones del proyecto.
61                 digitalWrite(LED3, LOW); //Ya que el usuario acertó, apagamos el LED que estaba encendido.
62                 presionado = true; //Cambiamos el estado de la variable que controla nuestro ciclo while para poder salir.
63             }
64             else errores++; //Si el LED que estaba encendido no era el 2, el usuario se equivocó. Agregamos al contador de errores.
65         }
66     }
67
68     //Pequeño ciclo while que ayuda a evitar que el programa lea el botón presionado múltiples veces y lo cuente como error:
69     while (digitalRead(boton1) == HIGH || digitalRead(boton2) == HIGH || digitalRead(boton3) == HIGH) {
70         delay(10);
71     }
72 }
73
74
```

6. Explicación del código

El código en Arduino usualmente se divide en dos secciones: “setup” y “loop”. En setup se pone el código que se ejecutará una vez únicamente al iniciar el sistema, y en loop el que se desea repetir mientras el Arduino esté encendido.

Primeramente se crearon tres variables a las cuales se les llamó “LED1”, “LED2” y “LED3”, y se les asignaron los números 11, 12 y 13, que son los puertos del Arduino donde cada uno está conectado respectivamente. También se crearon las variables “boton1”, “boton2” y “boton3”, y se les asignaron los números 2, 3 y 4. También fueron creadas las variables “opción”, “aciertos” y “errores”, buscando que en ellas se guarde cual es el LED seleccionado, cuántas respuestas correctas van y cuántas están mal. También se creó la variable “presionando”, la cual funciona para detectar cuando el usuario presiona correctamente el botón que corresponde.

En la sección de “setup” utilizamos la función “pinMode”, que configura los puertos donde están conectados los LED como “INPUT” (entrada) o “OUTPUT” (salida). También se usó la función randomSeed(analogRead(A0)), la cual genera números aleatorios mediante la lectura del puerto analogica A0.

Después en nuestra sección de “loop”, primero se usa random(1, 4) con la finalidad de generar aleatoriamente un número entre 1 y 3, y que este se almacene en “opción”. Ya con un número obtenido, se encenderá alguno de los LED mediante “digitalWrite” y el estado “HIGH”. Posteriormente, la variable “presionado” será establecida como “false” y mediante el “while” se mantendrá el sistema en espera hasta que el usuario presione el botón que corresponda al botón prendido. Con “digitalRead” se checa de manera constante en que estado se encuentra el botón, si este está presionado y corresponde al LED prendido, los aciertos se estarán incrementando. El LED será apagado utilizando el estado “LOW” y la variable “presionando” cambia a “true”, haciendo así que el programa siga con otra selección aleatoria nueva. Si el botón que se presiona no corresponde al LED que se encuentra prendido, aumentará la cantidad de errores, sin embargo, el LED seguirá encendido hasta que se presione el botón correcto. Finalmente se utiliza otro ciclo “while” para evitar que el programa lea demasiado rápido el estado actual del botón y lo marque como un error de la siguiente ronda porque el usuario no dejó de presionarlo a tiempo.

7. Resultados

Después de realizar las conexiones correspondientes y cargar el código en el Arduino, se hicieron distintas pruebas buscando observar si el sistema funciona correctamente. Pudo comprobarse que al momento de iniciar el programa, uno de los tres LEDs se enciende aleatoriamente y así permanece hasta que se presiona el botón que le corresponde, e igualmente, que si el botón presionado es incorrecto, el sistema permanece en su estado y el LED no se apaga hasta que el botón presionado sea correcto para así iniciar con la selección de un nuevo LED aleatoriamente.

LED prendido	Botón presionado	Resultado
LED rojo	Botón incorrecto	El LED permanece prendido
LED rojo	Botón correcto	El LED se apaga y prende otro aleatoriamente
LED azul	Botón incorrecto	El LED permanece prendido
LED azul	Botón correcto	El LED se apaga y prende otro aleatoriamente
LED verde	Botón incorrecto	El LED permanece prendido
LED verde	Botón correcto	El LED se apaga y prende otro aleatoriamente

8. Conclusión

En esta actividad logramos construir y programar un sistema de entretenimiento utilizando como herramientas un Arduino, tres LEDs y tres botones.

El programa logró que cada LED tuviera un botón en específico y que cada LED fuera seleccionado de una manera aleatoria de acuerdo a las pulsaciones de los botones. En las pruebas se logró verificar que, si el botón correcto correspondiente al LED es presionado, el LED se apaga y se selecciona uno nuevo al azar, e igualmente, que si el botón presionado no es el correcto, el LED seguirá encendido hasta que sea presionado el botón correcto, con lo cual se pudo cumplir con el objetivo de la actividad al llegar al funcionamiento esperado del programa.

9. Referencias

Boylestad, R. L., & Nashelsky, L. (2013). *Electronic devices and circuit theory* (11a ed.). Pearson Education.

SparkFun Electronics. (s.f.). *Pull-up resistors*. SparkFun Learn. Recuperado el 31 de agosto de 2026, de <https://learn.sparkfun.com/tutorials/pull-up-resistors/all>